UNIVERSITI MALAYSIA PERLIS

FAKULTI TEKNOLOGI KEJURUTERAAN ELEKTRIK

EMJ23204 MICROCONTROLLER SYSTEMS DESIGN

Project Title: Design and Implementation of a Traffic Light Control System Using Keypad Input and 7-Segment Display on Raspberry Pi Pico

NAME	**Adel Husham Mohamedain Yousuf**
MATRIC NUMBER	**231090057-1**
PC NUMBER	**No. 3**
GROUP	**G7**

1. Introduction

This LAB focuses on the design and implementation of a traffic light countdown system using Raspberry Pi Pico and CircuitPython. The main objective of this system is to control the timing of traffic light phases based on user input from a keypad. The system integrates both hardware and software components, where the keypad is used as an input device, the 7-segment display is used to show the countdown value, and three LEDs represent the traffic light states: green, yellow, and red.

When the push button (SW1) is pressed, the system allows the user to enter a countdown value between 0 and 99 using the keypad. After confirming the value, the system starts the traffic light sequence automatically. The green phase uses the user-defined value, while the yellow and red phases use predefined values. This project demonstrates important concepts such as digital interfacing, real-time control, and embedded system design.

2. Methodology

2.1 Flowchart

The flowchart in Figure 1 illustrates the overall operation of the traffic light control system. First, the Raspberry Pi Pico initializes all hardware components, including the LEDs, keypad, 7-segment display, and push button. The system then waits until the user presses the push button (SW1). After that, the user enters a countdown value (00–99) using the keypad and confirms it by pressing the # key. The entered value is saved as the green light duration. The system then starts the traffic light sequence by activating the green LED with the user-defined countdown, followed by the yellow LED for 3 seconds and the red LED for 5 seconds. Finally, the sequence repeats continuously using the same green countdown value.

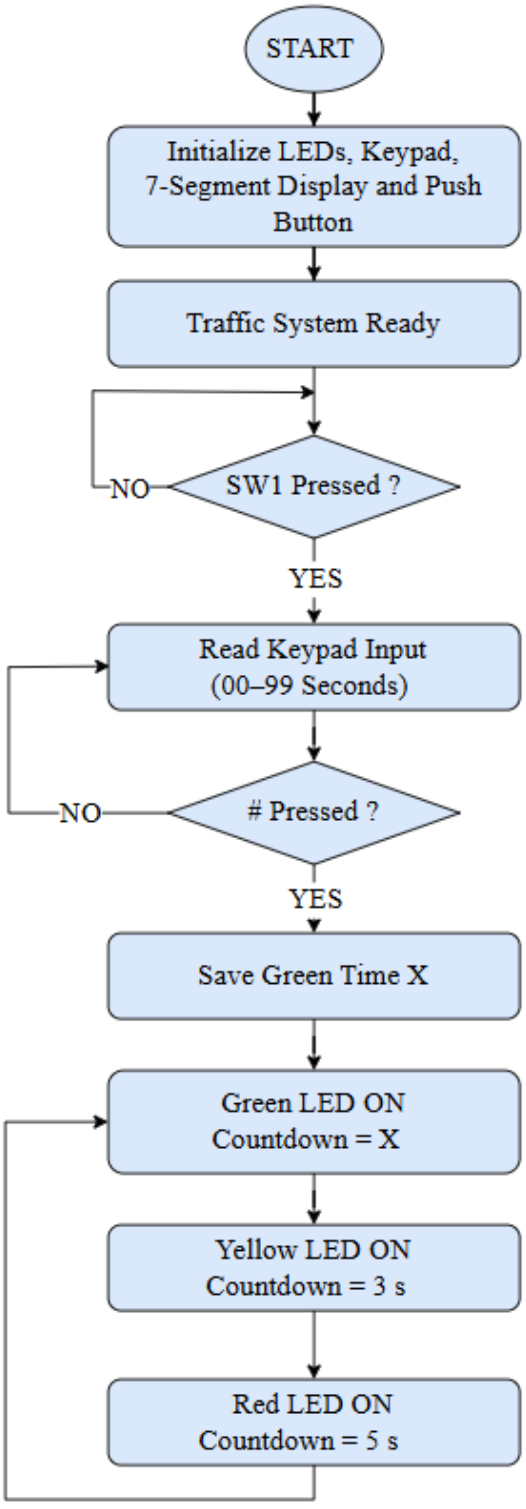

Figure 1: Flowchart of the traffic light control system using Raspberry Pi Pico, keypad input, and dual 7-segment display.

2.2 Full Coding

```
import board
import digitalio
import time

# ============================================================
# INITIALIZATION
# ============================================================

# Traffic LEDs
green_led = digitalio.DigitalInOut(board.GP26)
yellow_led = digitalio.DigitalInOut(board.GP27)
red_led = digitalio.DigitalInOut(board.GP28)

for led in [green_led, yellow_led, red_led]:
    led.direction = digitalio.Direction.OUTPUT
    led.value = False

# Push button (SW1)
sw1 = digitalio.DigitalInOut(board.GP15)
sw1.direction = digitalio.Direction.INPUT
sw1.pull = digitalio.Pull.UP

# ------------------------------------------------------------
# CD4511 (7-Segment BCD)
# A=GP5, B=GP4, C=GP3, D=GP2
# ------------------------------------------------------------
A = digitalio.DigitalInOut(board.GP5)
B = digitalio.DigitalInOut(board.GP4)
C = digitalio.DigitalInOut(board.GP3)
D = digitalio.DigitalInOut(board.GP2)

for pin in [A, B, C, D]:
    pin.direction = digitalio.Direction.OUTPUT

bcd_pins = [A, B, C, D]

# Latch pins
s72 = digitalio.DigitalInOut(board.GP7)   # Tens
```

```
s71 = digitalio.DigitalInOut(board.GP6)   # Ones

s72.direction = digitalio.Direction.OUTPUT
s71.direction = digitalio.Direction.OUTPUT

# ------------------------------------------------------------
# KEYPAD ENCODER (MM74C922)
# ------------------------------------------------------------
data_available = digitalio.DigitalInOut(board.GP14)
data_available.direction = digitalio.Direction.INPUT
data_pins = [board.GP10, board.GP11, board.GP12, board.GP13]

data_inputs = []
for pin in data_pins:
    dio = digitalio.DigitalInOut(pin)
    dio.direction = digitalio.Direction.INPUT
    data_inputs.append(dio)

# Mapping (correct)
keypad_map = {
    12: '1', 13: '2', 14: '3', 15: 'A',
    8: '4', 9: '5', 10: '6', 11: 'B',
    4: '7', 5: '8', 6: '9', 7: 'C',
    0: '*', 1: '0', 2: '#', 3: 'D'
}

# ============================================================
# FUNCTIONS
# ============================================================

def read_keypad():
    if data_available.value:
        value = 0
        for i, pin in enumerate(data_inputs):
            if pin.value:
                value |= (1 << i)
        return keypad_map.get(value, '?')
    return None

```

```
78. def send_bcd(value):
79.     for i in range(4):
80.         bcd_pins[i].value = (value >> i) & 1
81.
82. def latch(pin):
83.     pin.value = 0
84.     time.sleep(0.001)
85.     pin.value = 1
86.
87. def display_number(num):
88.     tens = num // 10
89.     ones = num % 10
90.
91.     send_bcd(tens)
92.     latch(s72)
93.
94.     send_bcd(ones)
95.     latch(s71)
96.
97. # ---------------------------------------------------------
98. # GET USER INPUT
99. # ---------------------------------------------------------
100. def get_user_input():
101.     value = ""
102.
103.     print("Enter countdown (0-99), press # to start")
104.
105.     while True:
106.         key = read_keypad()
107.
108.         if key:
109.             print("Key Pressed:", key)
110.
111.             if key.isdigit():
112.                 if len(value) < 2:
113.                     value += key
114.                     print("Current Input:", value)
115.
116.             elif key == "#":
```

```
                if value == "":
                    print("Error: No input")
                else:
                    return int(value)

            else:
                print("Ignored key")

            time.sleep(0.3)

# ------------------------------------------------------
# COUNTDOWN FUNCTION
# ------------------------------------------------------
def show_countdown(seconds, led, name):

    print(name, "PHASE START")

    # Turn OFF all LEDs first
    green_led.value = False
    yellow_led.value = False
    red_led.value = False

    led.value = True

    for i in range(seconds, -1, -1):
        print(name, "Time:", i)
        display_number(i)
        time.sleep(1)

    led.value = False

# ======================================================
# MAIN PROGRAM
# ======================================================

print("Traffic Light System Ready")

# Wait for SW1
while True:
```

```
156.     if not sw1.value:
157.         x = get_user_input()
158.         break
159.
160. # Fixed durations
161. y = 3   # Yellow
162. z = 5   # Red
163.
164. # Main loop
165. while True:
166.     show_countdown(x, green_led, "GREEN")
167.     show_countdown(y, yellow_led, "YELLOW")
168.     show_countdown(z, red_led, "RED")
169.
```

2.3 Hardware Setup

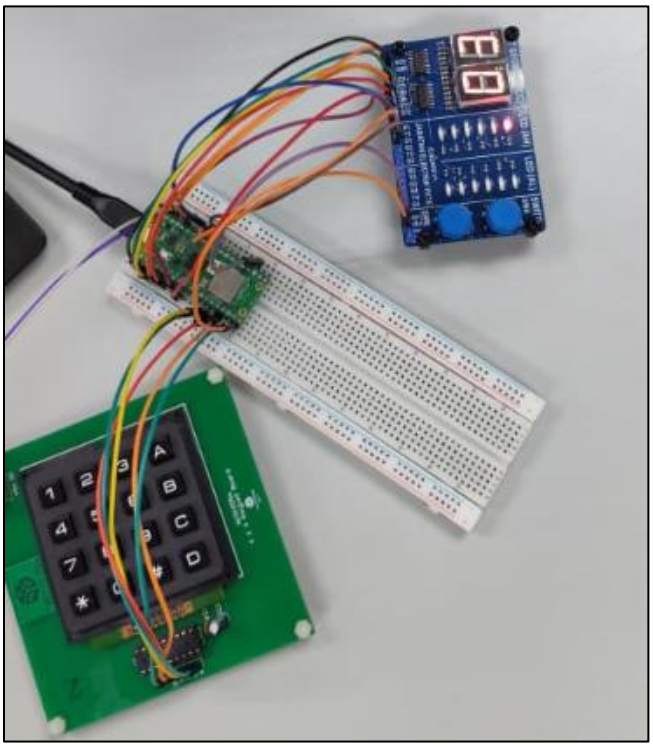

Figure 2: Hardware setup of the traffic light system using Raspberry Pi Pico, keypad, and 7-segment display.

As shown in Figure 2, the system consists of a Raspberry Pi Pico connected to a keypad with MM74C922 encoder, a 2-digit 7-segment display using CD4511 decoder, and three LEDs representing the traffic light. The keypad is used to enter the countdown value, while the encoder converts the key press into binary signals. The 7-segment display receives BCD signals from the Pico to display numbers. The LEDs are connected to GPIO pins and indicate the traffic light phases. The push button (SW1) is used to start the system.

2.4 Output Result

2.4.1 Green Phase

In this phase, the green LED turns ON, indicating that the system has started the traffic light sequence. The countdown begins from the value entered by the user using the keypad and decreases every second until it reaches zero.

The value is displayed clearly on the 7-segment display, allowing the user to monitor the remaining time. This phase confirms that the system can correctly read the user input and use it to control the duration of the green light.

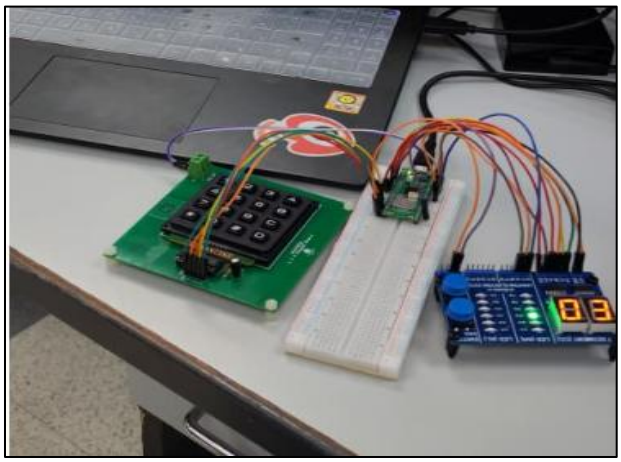

Figure 3: Green LED ON during user-defined countdown.

2.4.2 Yellow Phase

After the green phase is completed, the system automatically switches to the yellow phase. In this phase, the yellow LED turns ON to indicate a transition between green and red. The system performs a fixed countdown of 3 seconds, and the value is displayed on the 7-segment display. This phase shows that the system can move between different states automatically and follow predefined timing.

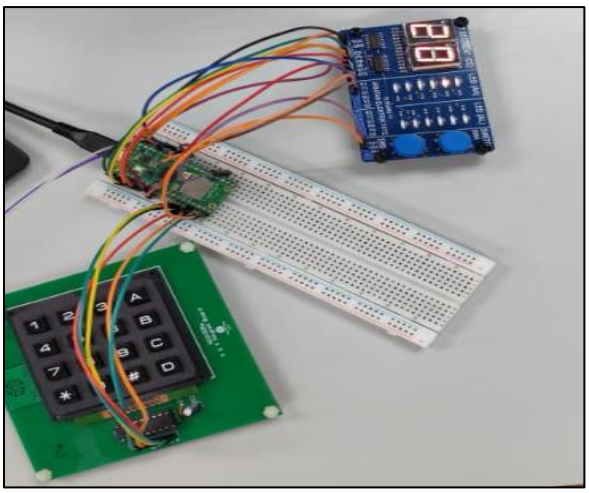

Figure 4: Yellow LED ON with fixed countdown.

2.4.3 Red Phase

After the yellow phase, the system enters the red phase. The red LED turns ON, indicating that the traffic must stop. The system performs a fixed countdown of 5 seconds, and the value is displayed on the 7-segment display until it reaches zero. This phase confirms that the system completes the full traffic light sequence correctly.

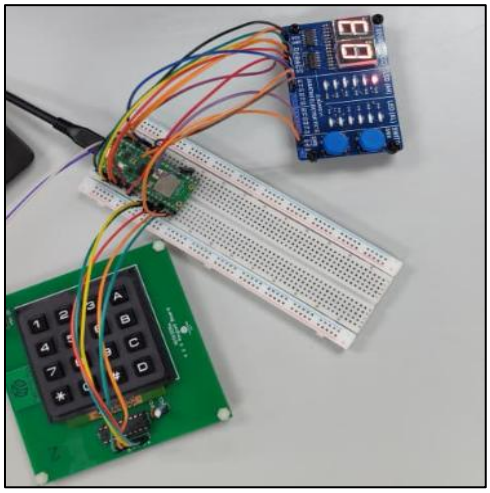

Figure 5: Red LED ON with fixed countdown.

2.4.4 Thonny Shell Output

In addition to the hardware output, the system also displays the operation in the Thonny Shell. It shows the current phase (GREEN, YELLOW, RED) and the countdown values for each second. This helps to monitor the system behavior and confirms that the program is working correctly in real time.

Figure 6: Output displayed on the Thonny Shell.

3. Conclusion

In conclusion, the traffic light countdown system was successfully implemented using Raspberry Pi Pico and CircuitPython. The system correctly reads the user input from the keypad and uses it to control the green light duration.

The yellow and red phases work using fixed timing, which makes the system simple and stable. The 7-segment display shows the countdown clearly, and the LEDs indicate the traffic light states.

This project helped in understanding how to connect hardware components such as keypad, LEDs, and display with software. It also improved knowledge about input reading, output control, and timing in embedded systems.

Overall, the system works as expected and meets all the required conditions. It is a good example of a simple embedded system that can be improved in the future by adding more features.